Bibli-Jeu2 : présentation du projet

Valentin et Yassin

Le projet

Bibli-Jeu2 est une application web, réalisée en binôme, qui permet de rechercher des jeux vidéo et de se constituer une bibliothèque personnelle. La recherche s'appuie sur l'API publique RAWG, une base de données de plus de 500 000 jeux. Les jeux ajoutés sont enregistrés dans une base de données et restent disponibles après un redémarrage de l'application.

Technologies utilisées : Java, Spring Boot, Thymeleaf pour les pages HTML, Spring Data JPA avec une base H2, JUnit pour les tests et Git avec GitHub pour le travail à deux. Le code est sur le dépôt gamatrick/Bibli-Jeu2.

Architecture

L'application est organisée en couches : chaque couche a un rôle précis et ne parle qu'à la couche suivante.

1. **Controllers** (`HomeController`, `BibliothequeController`) : reçoivent les requêtes du navigateur et renvoient la bonne page.
2. **Services** (`RawgService`, `BibliothequeService`) : contiennent la logique. `RawgService` interroge l'API RAWG, `BibliothequeService` gère l'ajout, la liste et la suppression des jeux.
3. **Repository** (`JeuRepository`) : accède à la base de données grâce à Spring Data JPA.
4. **Models** : `RawgJeu` et `RawgReponse` représentent le JSON reçu de RAWG, `Jeu` est l'entité enregistrée en base (nom, image, date de sortie, note, date d'ajout).

Les endpoints

L'application expose quatre adresses. Les actions qui modifient la bibliothèque utilisent POST, puis redirigent vers une page pour qu'un rafraîchissement (F5) ne renvoie pas le formulaire.

Adresse	Méthode	Rôle
`/` et `/?nom=zelda`	GET	Page d'accueil : formulaire de recherche et résultats sous forme de cartes
`/bibliotheque`	GET	Affiche les jeux enregistrés, du plus récent au plus ancien
`/bibliotheque/ajouter`	POST	Ajoute un jeu à la bibliothèque (refusé s'il y est déjà)
`/bibliotheque/{id}/supprimer`	POST	Retire un jeu de la bibliothèque

Côté API externe, l'application appelle (20 résultats maximum par recherche) `GET https://api.rawg.io/api/games?key=<clé>&search=<nom>&page_size=20`. La clé n'est jamais écrite dans le code : elle est lue dans la variable d'environnement `RAWG_API_KEY`.

Organisation du travail

Nous avons d'abord développé chacun, de notre côté, une version simple du site. Nous avons ensuite comparé les deux versions et choisi ensemble celle qui servirait de base au projet à rendre. Chacun

travaille sur sa propre branche (develop_val pour moi, dev-yas pour mon binôme). La version retenue a été intégrée à la branche principale master par une pull request, et les nouvelles fonctionnalités s'ajoutent sur cette base.

La valeur ajoutée de Valentin

J'ai mis en place le dépôt GitHub et codé les deux fonctionnalités principales, sur toutes les couches de l'application.

- **Dépôt GitHub** : création du dépôt gamatrick/Bibli-Jeu2, organisation en branches (master, develop_val, dev-yas) et intégration de ma branche dans master par une pull request.
- **Fonction de recherche** : appel de l'API RAWG avec `RestClient`, conversion du JSON en objets Java, affichage des résultats en cartes (titre, image, date de sortie, note).
- **Fonction bibliothèque** : ajout d'un jeu depuis les résultats, affichage de la liste, suppression, et blocage des doublons : un jeu déjà enregistré affiche « Déjà dans ta bibliothèque » dans les résultats au lieu du bouton d'ajout. J'ai aussi fait les deux pages HTML et la feuille de style.
- **Controllers, Services, Models, Repository** : j'ai écrit ces quatre couches, décrites dans la partie Architecture.
- **Tests unitaires** : quatre tests JUnit sur le service de bibliothèque (ajout d'un jeu, refus d'un doublon, champ vide enregistré comme vide, suppression). Ils utilisent une base en mémoire pour ne pas toucher aux vraies données. Un cinquième test vérifie que l'application démarre.

La valeur ajoutée de Yassin

J'ai enrichi l'interface et l'expérience utilisateur en implémentant les fonctionnalités suivantes :

- **Filtre pour affiner la recherche** : ajout de filtres dynamiques combinant les notes des jeux et un tri alphabétique fonctionnel (A-Z / Z-A) sur les résultats de recherche et la bibliothèque.
- **Modification du style** : refonte complète de la feuille de style pour moderniser l'interface et l'adapter aux pages HTML.
- **Genre des jeux** : extraction et affichage dynamique des genres de chaque jeu (action, RPG, FPS, etc.) à partir des données de l'API et de la base.